

Uncertainty-Aware Contextual Recognition of Sign Language Sequences

Hema Sravani Koshtu and Dheeraj Royal Galla

CS 5100: Foundations of Artificial Intelligence

Northeastern University

Abstract

In this study, we develop a recognition framework for WLASL that employs both uncertainty and contextual sequence decoding by fusing a CNN with an HMM. We fine-tune ResNet18 on WLASL for sign classification with a test set top-1 and top-5 accuracy of 56% and 88%, respectively. Our entropy-based approach finds that there is considerable uncertainty in our CNN predictions because of the highly similar appearance of many signs. By leveraging sign context through the Viterbi algorithm of an HMM trained on artificially created sign sequences, we increase our slot accuracy from 56.22% to 65.67% and our sequence-level accuracy from 15.33% to 49.67%. Qualitatively, we find that the HMM is correcting for the highest uncertainties with a mean entropy of 1.999 nats, which accounts for 86.8% of the maximum theoretical entropy, compared to the relatively lower 1.852 nats for already correct predictions. Our results illustrate how the benefit of contextual sequence decoding can be maximized when CNN predictions are more uncertain. Finally, we describe the REST API of our inference pipeline.

1. Introduction

The issue of sign language recognition is one that carries a great deal of importance within the field of artificial intelligence applications and is directly related to technology used for communication purposes. American Sign Language is the native language of approximately 250,000 to 500,000 individuals in the United States and Canada, but automatic recognition programs that could assist in facilitating communication are not widely available. Although most commercially available speech recognition software has been developed for speech languages, there exists a definite need for such software in the realm of signing languages.

The technical problem at the heart of ASL sign recognition stems from two sources of difficulty that compound each other. The first issue is that there are many ASL signs that have similar appearance, where, for example, the signs for “man” and “who,” or “orange” and “many” look very alike. This means that classifying frames in isolation becomes an ambiguous process. Secondly, classifiers that operate by looking at video clips individually, without being aware of the surrounding signs, neglect a wealth of additional information. In the same way that spoken languages use adjacent words to guide interpretation, so too does signed language.

This work tackles both problems by integrating visual deep learning feature extraction with sequence modelling through a probabilistic approach. After training of a fine-tuned ResNet18 convolutional neural network to generate per-frame probability distributions over the ten words in ASL, Shannon entropy is used as an uncertainty metric that captures the confidence of the model on the frame-level predictions. In turn, the Hidden Markov Model (HMM), which applies Viterbi decoding on the basis of estimated transition probabilities between words, allows for the reinterpretation of sequences of uncertain CNN outputs into sequences of actual words. The key scientific question concerns the benefits of contextual models for accurate recognition, especially among high-uncertainty cases.

The selection of HMMs to model sequences has been justified by a rich literature on the topic. Koller et al. (2016) illustrated that the combination of CNN and HMM yielded competitive performance on datasets for continuous sign language recognition due to the temporal smoothing ability of HMMs that

could not be provided by CNN alone. The proposed method in Koller et al. (2016) named as Deep Sign, employed convolutions as observations of HMM, which has been directly implemented in this paper. The WLASL dataset utilized for experiments has been published by Li et al. (2020). They achieved baseline results on the task of word-level sign language recognition and provided a large corpus of ASL videos crowdsourced from social media platforms.

The rest of the paper is organised as follows. The second section provides details of the methodology, including the data set, network architecture, training process, HMM structure, and experimental setup. The third section reports the findings. Section four provides the conclusions, limitations, and future directions. The references used in this study are provided in the final section.

2. Methods

2.1 Dataset

Word-Level American Sign Language (WLASL) dataset created by Li et al. (2020) was used for performing experiments. The WLASL is the largest public dataset on ASL word-level sign recognition that consists of several thousand video samples of signs corresponding to over two thousand glosses taken from publicly available sources on the Internet. For the current study, the `ns1t_100` sample was used that included the most frequently occurring signs in the dataset. Out of `ns1t_100` signs, 10 signs were chosen based on their availability and visual distinctiveness: basketball, birthday, but, city, man, many, orange, play, shirt, and who. Visual confusability of such pairs as man/who and orange/many was the criteria for choosing these signs because they provided a challenge for the HMM disambiguation task.

The raw video footage was divided based on word categories according to the WLASL dataset metadata, leading to between six and fourteen video footages for each category. For each video, preprocessing is done with OpenCV, and up to twelve frames per video footage are selected at uniform intervals within the range of the action segment defined from the `ns1t_100` dataset. This method of selecting frames guarantees that the selected frames belong to the signing action segment and not to the pre-signing or post-signing idle segments.

2.2 CNN Architecture and Training

Visual Classifier is a Convolutional Neural Network, ResNet18 architecture, which has been pre-trained on ImageNet and finetuned on WLASL Frames. ResNet18 has been chosen for being a good trade-off between its complexity and computational performance, as well as having its feature extractors generalizable to tasks related to the appearance of hands and bodies. The training involved unfreezing the two uppermost layers, which include layer3 and layer4 and the linear classifier head, whereas the lower feature extractors remained frozen. A Dropout layer with 0.5 probability was added prior to the linear classifier to mitigate overfitting on a relatively small data set.

In the training process, all the videos are considered to be composed of up to twenty frames. The model operates on all frames of a video at once, computing the feature vectors of size 512 by using the ResNet18 global average pooling layer, taking an average of all these vectors over time, and feeding them into the classifier. Temporal averaging helps in achieving this because it makes it possible for the model to utilise the entire information contained within the video while being computationally efficient enough.

The training was done for 40 epochs with the use of Adam Optimizer having a starting learning rate of 0.0005, weight decay of $1e-4$, and cosine annealing learning rate schedule with a minimum learning rate of $1e-6$. The cross-entropy loss with label smoothing of 0.1 was used to prevent overconfidence from small training dataset sizes. The experiments were performed on CPU.

2.3 Uncertainty Quantification

Entropy was used as a measure of uncertainty in the predicted probabilities from CNN. In the case of predicted probability distribution p across C classes, the Shannon entropy is computed as $H(p) = -\sum_c p(c) * \log(p(c))$ in units of nats. The maximum entropy that can be achieved when the distribution is uniform among ten classes in the vocabulary is $\ln(10) = 2.3026$ nats. Entropy on per-frame basis was computed using the probability distribution generated by applying softmax function on CNN prediction on each individual video frame, while clip level entropy was calculated as an average of all the frames' distributions.

2.4 HMM Sequence Modelling

The HMM was trained to capture contextual dependencies between successive signs using sequences of CNN clips as input. In this case, each position in the sequence is a hidden state and the CNN clip emissions represent observations. The transition matrix represents the probability of transitioning from one word category to another and was obtained using 300 generated sequences of signs. Since the WLASL corpus consists of isolated signs without any sentences, sequences of multiple words were created using clips sampled based on twelve pre-defined phrases such as "who, man, shirt" and "many, play, basketball". These phrases have some word bigrams that occur more frequently to give the transition matrix some structure to learn. Two different approaches for emitting probabilities were considered. In the first approach, called the average emission, the log-probability value is calculated as the logarithm of the mean value of the softmax probability over all frames in the video. In the second approach, the so-called confidence-weighted emission method is used, where each individual frame's softmax distribution is weighted with its highest predicted probability. This means that the frames with higher certainty will be assigned a greater weight.

Decoding of these hidden state sequences was done via Viterbi decoding, which selects the most likely hidden states sequence with respect to observation and transition matrices. These were four methods of decoding: greedy decoding (selecting the argmax per clip with no use of context), Hidden Markov Model with learned transitions and average emission probabilities, Hidden Markov Model with learned transitions and confidence-weighted emission probabilities, and Hidden Markov Model with fixed transitions.

2.5 REST API

A REST API for the Flask framework was implemented to serve the entire inference pipeline as a web application. The API includes three endpoints: one for checking the health of the model and returning information on its classes, one for making predictions from single clips and returning the top-3 predicted words with their confidence levels and entropy values, and another for making predictions from a set of clips and returning a Viterbi decoded sequence, along with

the greedy and HMM predictions for each slot. There are two other verification endpoints, which accept raw logit tensors and allow for exact comparison of the output of the API with the offline evaluation.

2.6 Evaluation Protocol

The evaluation process was conducted across three different levels. For isolated word recognition, top-1, top-3, and top-5 accuracies were calculated using the clip-level average-pooling predictions in addition to precision, recall, and F1-score for each class. In the case of sequence recognition, both the accuracy of slots (i.e., the accuracy of identifying individual word positions) and that of the entire sequence (i.e., accuracy of having all the words correctly recognized in the sequence) were used. The entropy correlation analysis was performed by dividing the testing samples into four groups (corrected by HMM, incorrect but correct CNN, worsened by HMM, both incorrect) and calculating the mean entropy within each category.

3. Results

3.1 CNN Training

The ResNet18 network trained for 40 epochs, and its optimum point was observed at epoch 38 where its loss was 11.89. Its training graph, illustrated below, begins at 54.48 at epoch 1, and then quickly declines until around epoch 15 to approximately 17, followed by a slow refinement period with some increases corresponding to the reduced learning rate using the cosine annealing strategy. Finally, it managed 56% accuracy for top-1, 76% for top-3, and 88% for top-5 for a dataset containing 25 samples. The difference between top-1 and top-3 accuracy can be explained by the similarity between classes because the predicted class is usually among the three top classes.

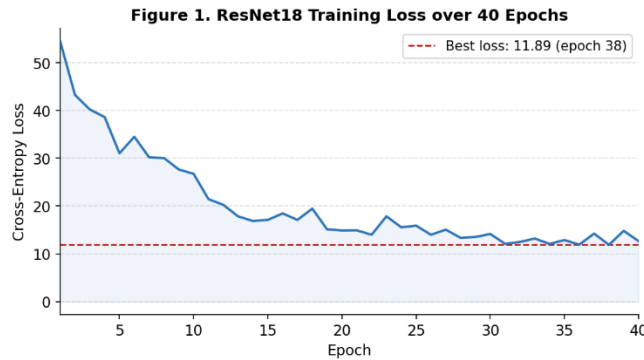


Figure 1. Training loss over 40 epochs. The dashed red line marks the best loss of 11.89 achieved at epoch 38.

The classification report per class shown in Figure 3 shows significant variation in recognition performance among the ten vocabulary terms. While the term “city” has an F1 score of 1.00 with both precision and recall, the term “birthday” does not have any correct predictions at all, having an F1 score of 0.00. High-performing vocabulary terms like “shirt” and “but,” which have F1 scores of 0.80 and 0.67 respectively, have unique hand shapes, while low-performing vocabulary terms such as “birthday” and “many,” which have F1 scores of 0.00 and 0.33 respectively, are often confused with signs that look alike.

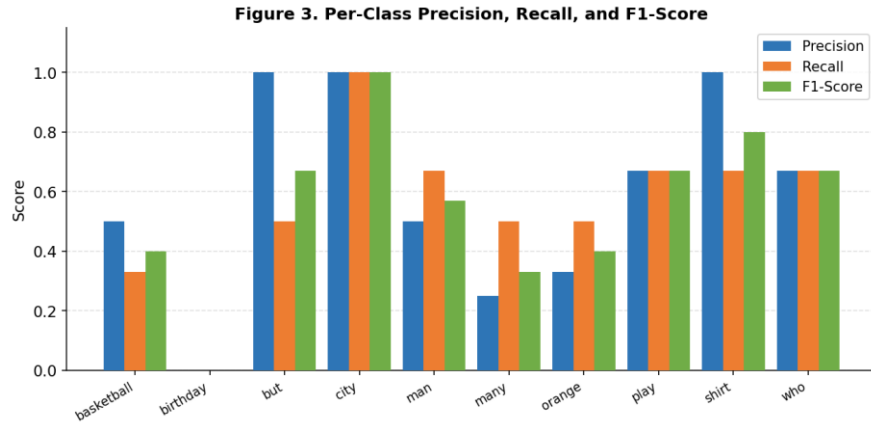


Figure 3. Per-class precision, recall, and F1-score for the ten ASL vocabulary items.

3.2 Uncertainty Analysis

An analysis of the Shannon entropy of the CNN predictions showed consistently high levels of prediction uncertainty for all frames. Per-frame entropy averaged at 1.29 nats, while the clip-level entropy averaged at 1.92 nats, when the maximum possible value for entropy was 2.30 nats for a uniform distribution across ten classes. The figures above demonstrate that the network was performing far below the maximum possible certainty level of the predicted outcomes due to the complexity of visual recognition of ASL signs on video clips. The fact that the entropy delta value was negative (-0.63) also confirmed that entropy decreased during the process of aggregation.

3.3 HMM Sequence Decoding

The decoding outcomes of HMM sequence are captured in Fig 2 & Table 1 below. Decoding using greedy approach, whereby decoding each clip separately, yields an accuracy rate of 56.22% at slot level and 15.33% at sequence level. Low sequence accuracy compared to slot accuracy can be

expected since the per slot error rate multiplies with increase in length of sequences.

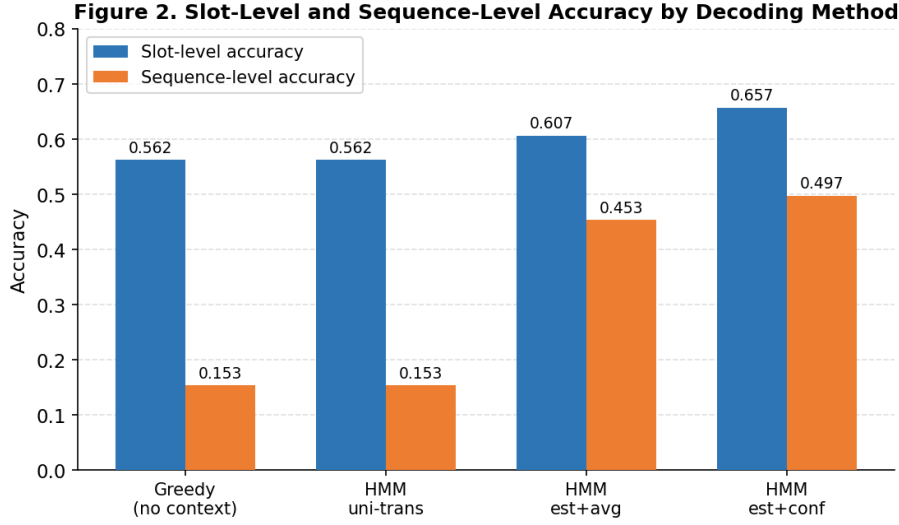


Figure 2. Slot-level and sequence-level accuracy for four decoding conditions.

The addition of an HMM with a uniform transition probability model (ablation condition) resulted in no gains, yielding identical values of 56.22% and 15.33% for slot accuracy and sequence accuracy respectively. This outcome supports the assertion that the transition probabilities have a greater impact on the success of HMM decoding than simple smoothing does. With the uniform transition probabilities, there is no bias towards any word sequence since the model is reduced to simply greedy decoding because the uniform transition model favors all words equally. On the other hand, using estimated transition probabilities and averaged emissions improved slot accuracy by +4.45pp (to 60.67%) and sequence accuracy by +30.00pp (to 45.33%). The most successful experiment involved confidence-weighted emissions which yielded the highest slot accuracy of 65.67% (+9.44pp from greedy) and 49.67% sequence accuracy (+34.33pp).

Decoding Method	Slot Acc.	Seq Acc.	Slot	Seq
Greedy	56.22%	15.33%	-	-
HMM-uniform transitions	56.22%	15.33%	+0.00%	+0.00%
HMM-est. trans. + avg emit.	60.67%	45.33%	+4.45%	+30.00%
HMM-est. trans. + conf. emit.	65.67%	49.67%	+9.44%	+34.43%

Table 1. Sequence recognition accuracy for all decoding conditions (300 sequences, 900 total word slots).

3.4 Qualitative Entropy Correlation

In order to verify whether contextual modeling yields maximum utility where the CNN is least sure about its output, the word slot of the test corpus was classified into four groups according to their outcome, that is, when the HMM had corrected a mistake made by the CNN (11 occurrences), both were right (13 occurrences), when the HMM had made a mistake in the right predictions of the CNN (14), and when both made a mistake (32).

Figure 4. Outcome Counts and Mean Prediction Entropy by Decoding Result

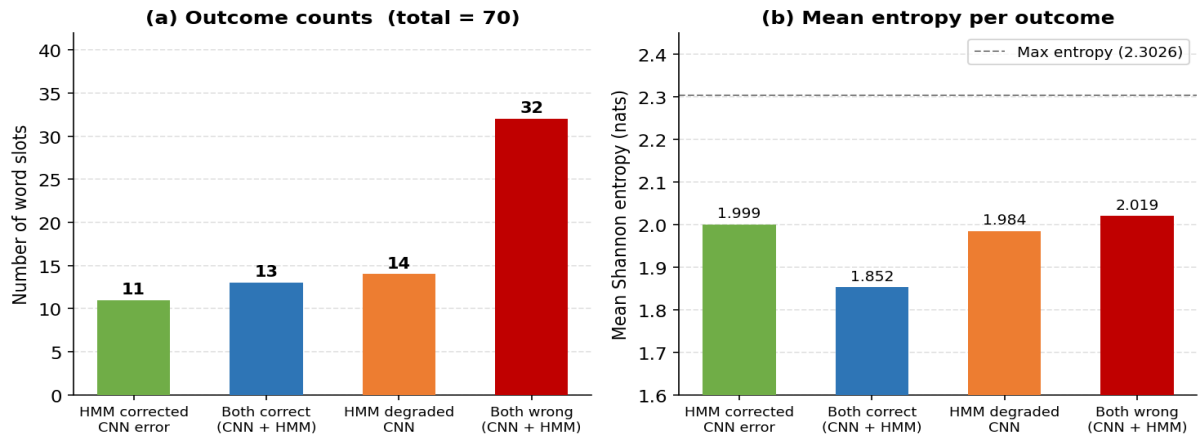


Figure 4. (a) Number of word slots in each outcome category and (b) mean Shannon entropy per category. Higher entropy indicates greater CNN uncertainty.

Mean entropy was found to be 1.999 nats (86.8% of maximum) in relation to the HMM-corrected slot samples. Comparatively, samples in which predictions were made accurately by CNN and then

corrected by HMM showed a mean entropy of 1.852 nats (80.4%). The difference of +0.147 nats supports the hypothesis that HMM makes corrections on highly uncertain and high entropy predictions of CNN. The qualitative analysis shows several representative samples of the process. For example, the sample of a clip labelled "man" had an entropy of 2.166 nats, and the confidence of CNN prediction was 19.2%. In this case, CNN had incorrectly predicted "who", but the HMM prediction turned out to be accurate due to transition context. A similar case happened with another clip labelled "but", where again the entropy was 2.160 nats, CNN made an incorrect prediction, and HMM made an accurate prediction.

It is necessary to openly acknowledge the 14 instances in which HMM interfered with the accurate CNN prediction. In all cases, the word "man" appeared. The HMM misinterpreted the word "man" as "orange" or "who," even though the CNN predicted the right answer correctly. A closer look at the transition matrix indicates that these mistakes are made due to the existence of template bigrams in which "man" appears in front of other class types with high probability. Therefore, even when the CNN predictions seem to be relatively accurate, the HMM will choose transitions based on the probabilities found during the training process because of the artificiality of training sequences.

4. Conclusions

It can be concluded from this study that the combination of uncertainty-aware context modeling contributes to enhancing the performance of sign language recognition compared to frame-by-frame CNN classification models. There are three main findings. First, a fine-tuned ResNet18 classifier shows high performance in isolated word classification with 56% top-1 and 88% top-5 accuracies on the ten-class ASL vocabulary, which proves that transfer learning from ImageNet is a successful way to start sign recognition. Second, using a Viterbi HMM with transition probabilities estimation and confidence scores results in higher performance of sequence recognition with 9.44 percentage points increase in slot accuracy and 34.33 percentage points increase in sequence accuracy. Third, the entropy correlation analysis provides evidence that contextual correction occurs mainly when uncertainties are high.

The method has several crucial weaknesses. Firstly, the vocabulary of 10 signs is extremely small compared to the number of signs needed to make ASL communication possible, given that real-life vocabularies involve thousands of signs. Secondly, the multiword phrases used in HMM training are artificial phrases, generated using templates, while no natural continuous signing was involved in learning the transition matrix. Thirdly, the test dataset consists of 25 samples, which makes individual variations of samples relevant to reported accuracy levels. Finally, the ResNet18 structure takes each frame individually, which eliminates the possibility of capturing temporal characteristics of hands' motion.

As compared to the architecture proposed by Koller et al. (2016), the main architectural difference is related to training. Specifically, the architecture of Deep Sign relied on a fully discriminative HMM training process where parameters of both the CNN and the HMM were trained simultaneously. In contrast, the current architecture separates CNN training from HMM training. Joint training would most certainly result in better performance since the CNN will be able to generate more appropriate emission distributions for use in the HMM. Additionally, the study of Li et al. (2020) indicates that more appropriate 3D models or graph skeleton models can be applied to capture temporal dependencies.

There are several ways the algorithm could be enhanced with practical benefits. The use of videos containing real sign language would have led to a better transition matrix. Testing the model using 50 or 100 classes would test the scalability of the approach. The inclusion of MediaPipe skeletal landmarks as another input channel for the CNN would improve the hand shape data. Joint CNN-HMM learning would reduce the disparity between the offline algorithm and the API implementation. The REST API developed in this research is a useful basis for these improvements, allowing the inference process to be tested and applied without changing the underlying recognition code.

In conclusion, this work shows how the application of deep visual features, uncertainty quantification through Shannon entropy, and probabilistic contextual sequence modeling using HMM decoding can lead to an SLR system that outperforms a simple frame-based baseline in terms of robustness and honesty, and provides a more interpretable approach to recognition than the

frame-based approach. The finding that contextual modeling contributes the most to the recognition of high-entropy samples represents a contribution that can be applied more broadly to the field of artificial intelligence, where estimating and dealing with uncertainties is vital.

5. Citations

- [1] [O. Koller, S. Zargaran, H. Ney, and R. Bowden, "Deep Sign: Hybrid CNN-HMM for Continuous Sign Language Recognition," in Proc. British Machine Vision Conference \(BMVC\), 2016.](#)
- [2] [Li, D., Rodriguez, C., Yu, X., and Li, H. \(2020\). Word-Level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision \(WACV\).](#)
- [3] [He, K., Zhang, X., Ren, S., and Sun, J. \(2016\). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition \(CVPR\), pp. 770-778.](#)
- [4] [Viterbi, A. J. \(1967\). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. IEEE Transactions on Information Theory, 13\(2\), 260-269.](#)
- [5] [Shannon, C. E. \(1948\). A mathematical theory of communication. Bell System Technical Journal, 27\(3\), 379-423.](#)
- [6] [Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. \(2009\). ImageNet: A Large-Scale Hierarchical Image Database. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition \(CVPR\), pp. 248-255.](#)